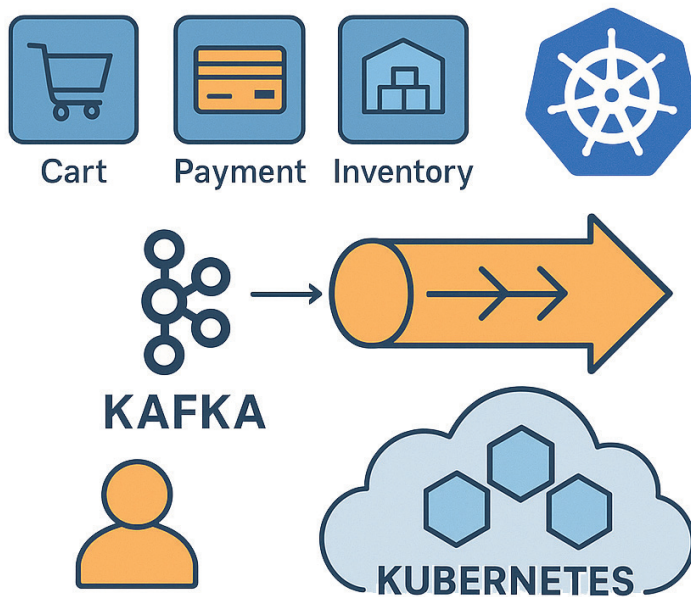


Building a Scalable E-Commerce Platform with Microservices, Kafka, and Kubernetes



This real-world implementation of a full-stack, microservices-based e-commerce platform uses open source technologies. The project demonstrates how open source tools can power scalable, reactive applications with production-grade architecture and provides insights into containerised deployments, observability, and service communication patterns.

In the modern digital ecosystem, building scalable and maintainable applications demands more than just writing code—it requires smart architecture, efficient communication between services, and seamless deployment pipelines. Microservices architecture has emerged as the go-to solution for developing complex, modular systems that can evolve rapidly without breaking the whole application.

This article dives into the development of a full-stack e-commerce platform built using open source technologies. From backend services written in Node.js and Express, to Kafka-powered asynchronous communication, to a containerised deployment on Kubernetes using Minikube—every layer is designed to reflect a real-world, production-grade microservices environment.

The frontend, built using Next.js, is tightly integrated with Clerk.dev for authentication and interacts with backend services through REST APIs and WebSockets for real-time responsiveness. This project serves as a hands-on blueprint for developers looking to build distributed, event-driven

systems using open source tools—offering practical insights into architecture, deployment, and scalability.

System architecture

The architecture of the platform is based on a modular microservices approach, where each functional unit is isolated into an independent service. Figure 1 shows system architecture of the e-commerce platform. This design promotes scalability, ease of maintenance, and rapid deployment.

Each microservice—Product, Order, Wishlist, Chat, and Notification, is responsible for a single domain and communicates with other services through a hybrid model:

- **Synchronous communication** using REST APIs for immediate, request-response interactions (e.g., Wishlist fetching product details).
- **Asynchronous communication** using Apache Kafka, allowing services to publish and consume events decoupled from one another (e.g., Order service publishing to a notification topic).